

CMSC 603

High-Performance Distributed Systems

Slurm for HPC clusters



Dr. Preetam Ghosh
Professor

Department of Computer Science
pghosh@vcu.edu

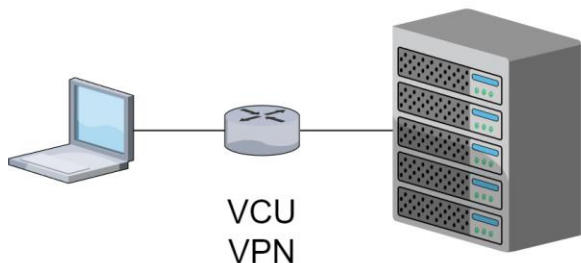
VCU HPC servers:

- **maple.cs.vcu.edu**
- **athena.hprc.vcu.edu**

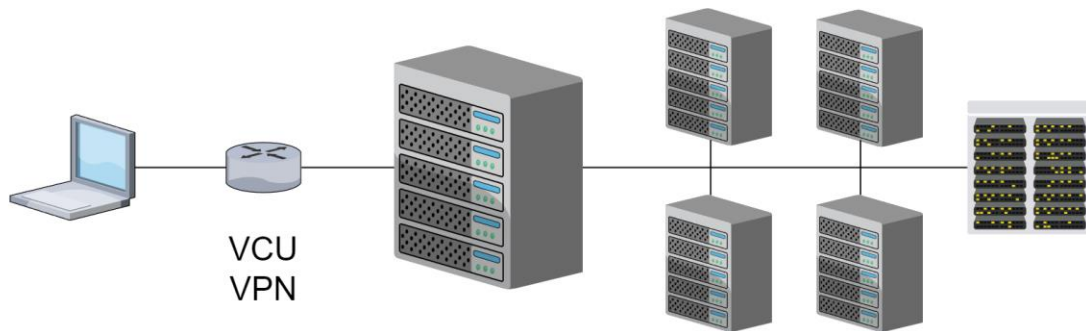
Single-node server to develop and test your code

Cluster to deploy your code. Requires Slurm to run

See <https://wiki.vcu.edu/display/unix/Athena+cluster>



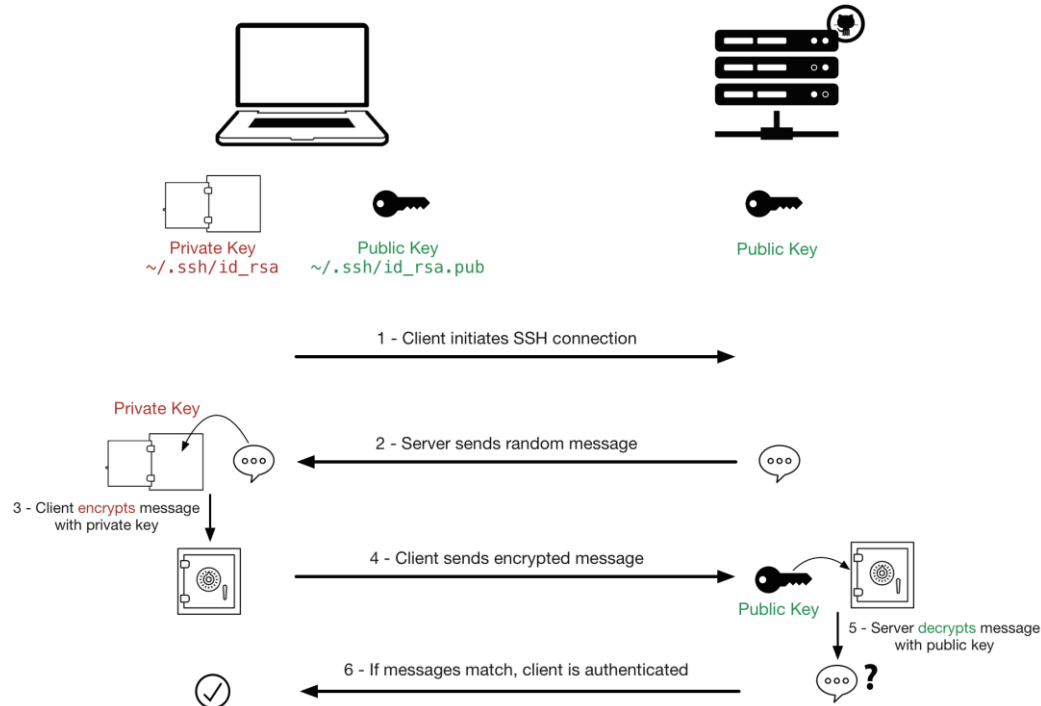
maple.cs.vcu.edu
cpus: 28
memory: 384 GB
gpu: NVIDIA 1080



athena.hprc.vcu.edu
nodes: 118
cpus: 7,564
memory: 42 TB
gpu: 34 (2 x P100, 8 V100, 8 A100, 16 H100)
3 PB network shared storage

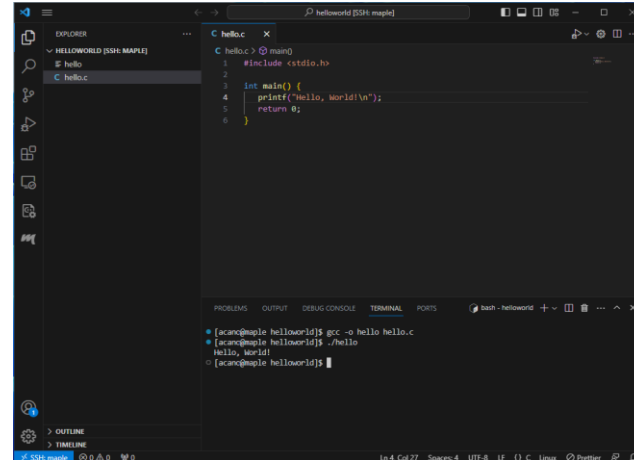
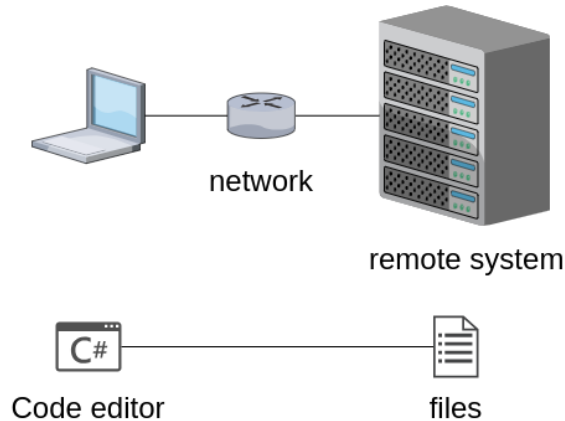
SSH passwordless login for maple and athena

- See instructions on Canvas for generating ssh keys and add them to the server



Visual Studio code for remote development on **maple.cs.vcu.edu** only

- VS code is a lightweight and powerful IDE that allows you to write, compile, and run source code on a remote system without having to install any tools locally
- See instructions on Canvas to set up VS Code on maple (after SSH keys setup)



The screenshot shows the Visual Studio Code interface with a remote connection to 'helloworld [SSH: maple]'. The Explorer panel on the left shows the file structure with 'helloworld [SSH: MAPLE]' expanded, containing 'hello' and 'hello.c'. The main editor displays the contents of 'hello.c', which is a C program that prints 'hello, world!'. The terminal at the bottom shows the execution of the program:

```
[sac@maple helloworld]$ gcc -o hello hello.c
[sac@maple helloworld]$ ./hello
hello, world!
[sac@maple helloworld]$
```

Visual Studio code for remote development on **athena.hprc.vcu.edu** only

- Use Open OnDemand at <https://athena3.hprc.vcu.edu>
- Delete the session when you finish to release resources for other users

VS Code

This app will launch a **VS Code** server web UI.

Partition

cpu

Number of CPUs

1

Only the code written to run on multiple CPUs can take advantage of more than 1 CPU.

Memory (GB)

8

GPU mode

shared

If using a GPU partition, select the GPU on shared mode (GUIs, development, small workloads) vs exclusive mode (all GPU and memory for this job).

Number of hours until closing this session automatically

4

Path to the workspace

☐ I would like to receive an email when the session starts

Launch

VS Code (500868)

1 node

1 core

Running

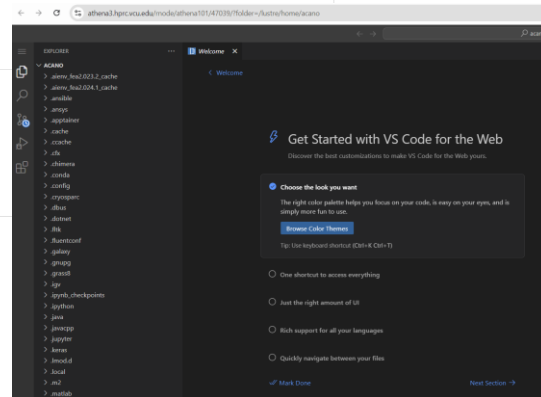
Host: >_athena101

Created at: 2024-08-27 08:53:23 EDT

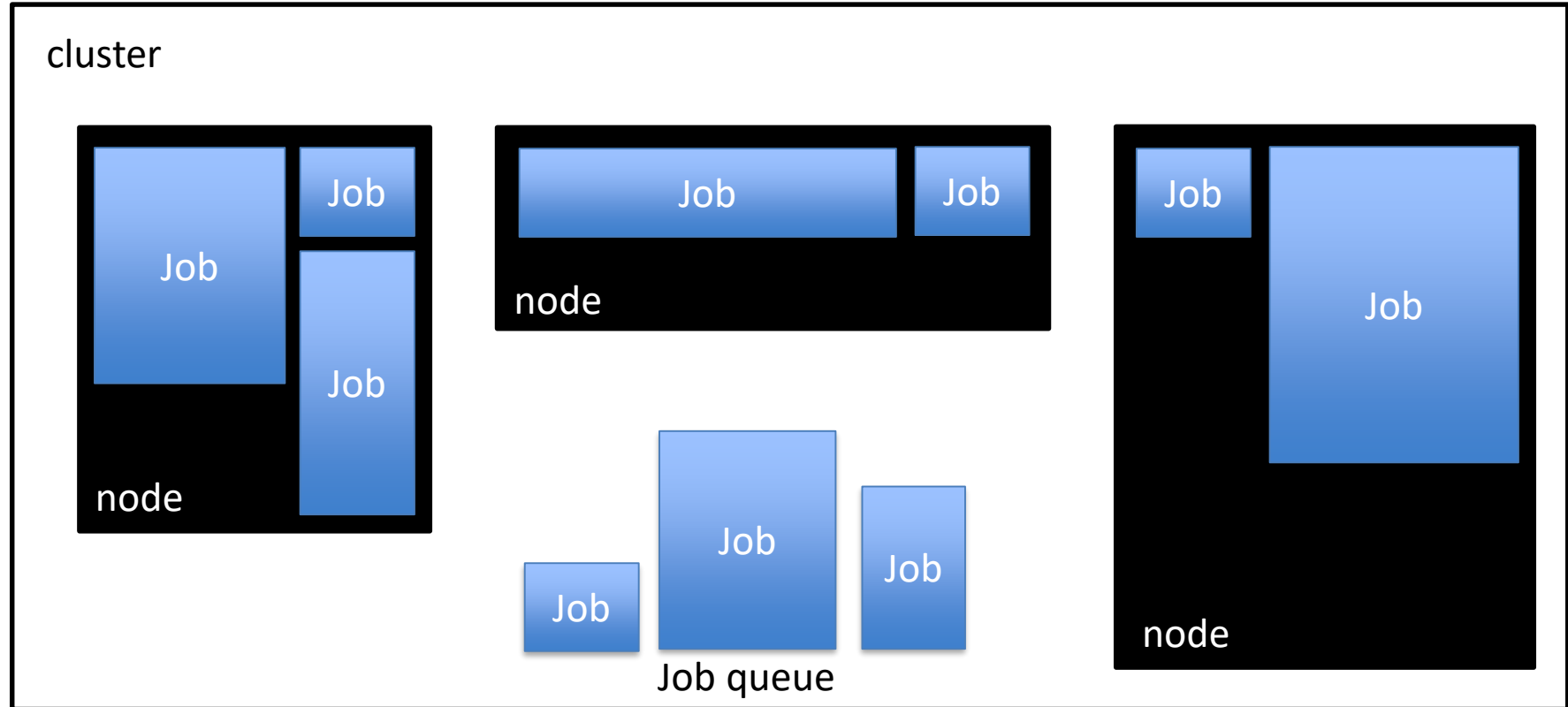
Time Remaining: 3 hours and 59 minutes

Session ID: 1a9e52c7-e16f-448c-9ddd-07ffbb5c365d

When you finish your session please click on the **Delete** button.



How job allocations on a cluster work: reservation of resources (CPUs, memory, GPUs)



Slurm (Simple Linux Utility for Resource Management):

- Cluster manager and job scheduler
- Allocates resources for running jobs and manages jobs queue
- **Node**: resources: CPUs, memory, GPUs
- **Partition**: groups of nodes with job/access restrictions
- **Job**: ID, name, time limit, quality of service, state, etc
- Basic commands:
 - **sinfo** - shows the partitions/nodes state (also **sinfo --Node -long**)
 - **squeue** - shows all jobs in the queue (also **squeue -u username**)
 - **sbatch <scriptname>** - launches a batch script
 - **scancel <jobID>** - cancels a job (also **scancel -u username**)
 - **srun** - runs a simple job



Slurm directives

Directive	Short	Long	Description
Partition	-p pname	--partition pname	Selects a partition for the job
Node count	-N 4	--nodes 4	Selects the number of nodes (4)
Node list	-w athenaXXX	--nodelist athenaXXX	Selects the nodes on which to run
CPUs	-c 4	--cpus-per-task 4	Number of CPUs per task (4)
Memory		--mem 8G	Total memory (M/G)
Job name	-J jname	--job-name jname	Provide a job name
Stderr	-e errorfile.out	--error errorfile.out	Save stderr to a file
Stdout	-o outputfile.out	--output outputfile.out	Save stdout to a file

Run a simple job in Slurm

- Example of a simple job to run the command “hostname” on the allocated node with default parameters (cpu partition, 1 node, 1 CPU, 16G memory)

```
srun
```

```
  [--partition cpu]
```

```
  [--nodes 1]
```

```
  [--cpus-per-task 1]
```

```
  [--mem=16G]
```

```
hostname
```

- Example of a simple job to run a python program with parameters specified by the user

```
srun
```

```
  --job-name myprogram
```

```
  --partition cpu
```

```
  --cpus-per-task 16
```

```
  --mem 8G
```

```
  --o output.log
```

```
python myprogram.py
```

Run an interactive job in Slurm

- Example of an **interactive** job. Allocates the resources and opens a new bash session on one of the granted nodes. Resources stay allocated till closing the session. Do **NOT** keep idle bash sessions consuming resources.

```
srun --partition cpu --cpus-per-task 4 --mem 4G --pty bash
```

- Running on the GPU partition (exclusive vs shared mode)

```
srun --partition gpu --gres gpu:1 --cpus-per-task 2 --mem 16G --pty bash
```

```
srun --partition gpu --gres shard:1 --cpus-per-task 4 --mem 32G --pty bash
```

Run a batch job in Slurm

- Example of a simple **batch** job. Allocates the resources and runs the commands.

```
#SBATCH --job-name HelloWorld
#SBATCH --partition cpu
#SBATCH --cpus-per-task 1
#SBATCH --mem 1G
#SBATCH --error %x.%j.error
#SBATCH --output %x.%j.output
#SBATCH --mail-type=ALL
#SBATCH --mail-user=myvcueid@vcu.edu

echo "Job begins"
module load anaconda3
conda activate venv
python myprogram.py
echo "Job ends"
```

Run as:

```
sbatch script.sh
```

The job will be added to the queue and run as soon as resources are available

Recommendations

- Nodes reserve 2 CPUs and 8G memory for the Operating System.
CPUs: use up to 126 CPUs per node
Memory: 242G / 494 G / 998G maximum
- Use the resources you need. Do not over allocate resources
- Bigger jobs will likely sit in the queue for longer. Runtime trade-off.
- Small serial code will likely run faster on your computer!
- Release resources when you are finished
- Do not create a large number of small files on the cluster
- Delete files when no longer needed
- Use /tmp for fast I/O local to the node
- Use centrally-installed software, particularly conda modules
- When in doubt, ask!

CMSC 603

High-Performance Distributed Systems

Slurm for HPC clusters



Dr. Preetam Ghosh
Professor

Department of Computer Science
pghosh@vcu.edu